

BoltAssist — AI Insurance Assistant on AWS

An AI assistant that answers insurance questions using the customer's own policy documents. Built on AWS — a voice & chat call center, an event-driven API, and the whole setup written as Terraform code.

BoltAssist answers customer insurance questions using only the company's policy files — an approach called RAG (retrieval-augmented generation). I built two systems on AWS: a voice/chat assistant and an event-driven API. Then I wrote the whole setup as Terraform code and ran the full cycle — **create** → **import** → **destroy** → **recreate**.

Architecture

System 1 — Voice/chat assistant

Customer (phone/chat) → Amazon Connect → Lex V2 (4 intents) → Lambda (container) → **LangChain agent**

- **Search:** HuggingFace embeddings + ChromaDB (policy files in S3)
- **Answer:** Amazon Bedrock — Claude Sonnet 4.6 → Polly (voice)

The bot answers only from the documents and shows its sources. If it can't answer, it passes you to a human agent.

System 2 — Event-driven API

Client → API Gateway

- **POST /ask** → ALB → ECS Fargate (always-on RAG API) — *fast, real-time*
- **POST /claims** → SQS → Lambda → SNS; failed messages → DLQ — *background*

I chose the right compute for each job: Fargate for the always-on API, Lambda for short background tasks.

Key Problems I Solved

- **Amazon Bedrock models:** Newer Claude models can't run directly — they need an inference profile. I replaced an old model that AWS shut down and used the global profile (the only one that works from Seoul), then set the right IAM permissions.
- **One image, two ways to run it:** the same Docker image runs as a Lambda function and as a web server (uvicorn) on Fargate. I changed the startup command so Fargate runs the web server.
- **Careful IAM permissions:** I split the roles — one to pull the image and write logs, one for the app to call S3 and Bedrock. Each role gets only what it needs.
- **ECS + ALB networking:** I connected the Fargate tasks to the load balancer, matched the network zones, and reused the listener. This fixed a 503 error that happened because no targets were connected.
- **Event-driven design:** SQS queues split up the work and retry on failure, with a DLQ to hold bad messages. SNS sends notifications. API Gateway sends straight to SQS, so no extra Lambda is needed.

Infrastructure as Code (Terraform)

- I rewrote the hand-built setup as Terraform code, and tested it safely on a separate copy first.
- I brought the existing resources into Terraform with **terraform import**, then fixed the small differences — auto-generated IAM names, managed vs inline policies, and Lex's special import-ID format.
- I ran the full cycle: I **deleted the whole stack and rebuilt it from code**. I solved an ordering problem with ECR — the image must exist before the function (make the repo → push the image → apply the rest) — and used `force_delete` to remove it cleanly.

Tech Stack

AWS Bedrock · Lambda · Lex V2 · Connect · Polly · ECS Fargate · ALB · API Gateway · SQS · SNS · S3 · ECR · IAM · CloudWatch

App LangChain (agent + tools) · ChromaDB · HuggingFace embeddings · FastAPI

Infra Docker · Terraform

Skills RAG / LLM apps · AWS architecture · Serverless & containers · Event-driven systems · Infrastructure-as-Code (Terraform) · IAM & security · Debugging